CAPSTONE PROJECT



SPACE X
FALCON 9
LANDING PREDICTION

Predicting first-stage booster recovery with data science

Prepared by | Shravani Pawar

IBM Data Science Professional Certificate

Applied Data Science Capstone

18 October 2026

From launch history to landing prediction

A complete data science workflow for a practical launch-planning question

Business question

Can historical launch details help estimate whether a Falcon 9 first-stage booster will land successfully? Reuse matters because recovered hardware can be refurbished for another mission.

Approach

Collect structured data from the SpaceX API and supplementary launch-history tables; clean and explore it with Pandas/SQL; communicate patterns with Folium and Plotly Dash; train and evaluate classifiers.

Main takeaway

Launch site, mission orbit, payload mass and flight history show useful patterns. In the standard course workflow, Decision Tree has the strongest cross-validation score among the four tested classifiers.

PROJECT PIPELINE

01 Collect — 02 Clean — 03 Explore — 04 Visualize — 05 Model — 06 Evaluate

Dataset snapshot: 90 Falcon 9 launches in the course API dataset. Results are historical associations, not guarantees for future launches.

Why predict a first-stage landing?

Problem framing, audience, and project objectives

Business context

Falcon 9's first stage is designed to return and be reused when mission conditions allow. Knowing whether a recovery is likely helps a launch-service analyst reason about mission risk and cost. A competitor can use the analysis to understand factors linked with a recoverable launch.

Prediction question

Given information available before or during launch planning - such as launch site, payload mass, orbit and flight sequence - can a model classify the landing outcome as successful or unsuccessful?

PROJECT OBJECTIVES

- Build a reproducible dataset from API data and web-scraped launch history.
- Investigate outcome patterns using visual analysis and SQL queries.
- Use a geographic map and an interactive dashboard to communicate results.
- Compare Logistic Regression, SVM, Decision Tree, and K-Nearest Neighbors (KNN).

Scope note: The model supports analysis and planning. It cannot replace launch engineering judgement, and some unsuccessful landings are deliberate mission choices.

Data collection: two complementary sources

A structured API dataset plus historical context from launch-history tables

Source A | SpaceX REST API

1. Send an HTTP GET request to the launch endpoint.
2. Parse the JSON response.
3. Use **pandas.json_normalize()** to flatten nested JSON fields into tabular columns.
4. Filter records to Falcon 9 and select fields relevant to launch and landing analysis.

Source B | Launch-history webpage

1. Request the Falcon 9 launch-history page.
2. Parse the response using **BeautifulSoup**.
3. Find table headers and launch rows in the HTML.
4. Convert the extracted table into a pandas DataFrame and standardize the fields before combining it with API data.

90
API dataset rows

3
Launch sites

4
Model families

Libraries: requests, pandas, NumPy, datetime, BeautifulSoup, and regular expressions as required by the scraping steps.

Data lineage matters: record the source URL, extraction date, column meanings and filters. API and Wikipedia tables can cover different time ranges, so the final dataset must be checked for duplicates and inconsistent labels.

Data wrangling: make the data model-ready

Cleaning and feature engineering before exploratory analysis and classification

01 Inspect

Shape, column names, data types, duplicates and missing values.

02 Filter

Keep Falcon 9 launches and remove irrelevant or leaking fields.

03 Clean

Normalize labels; convert dates, numbers and category values consistently.

04 Target

Create a binary landing label: 1 = successful landing; 0 = not successful.

05 Prepare

Impute missing numeric values as specified in the lab; encode categories.

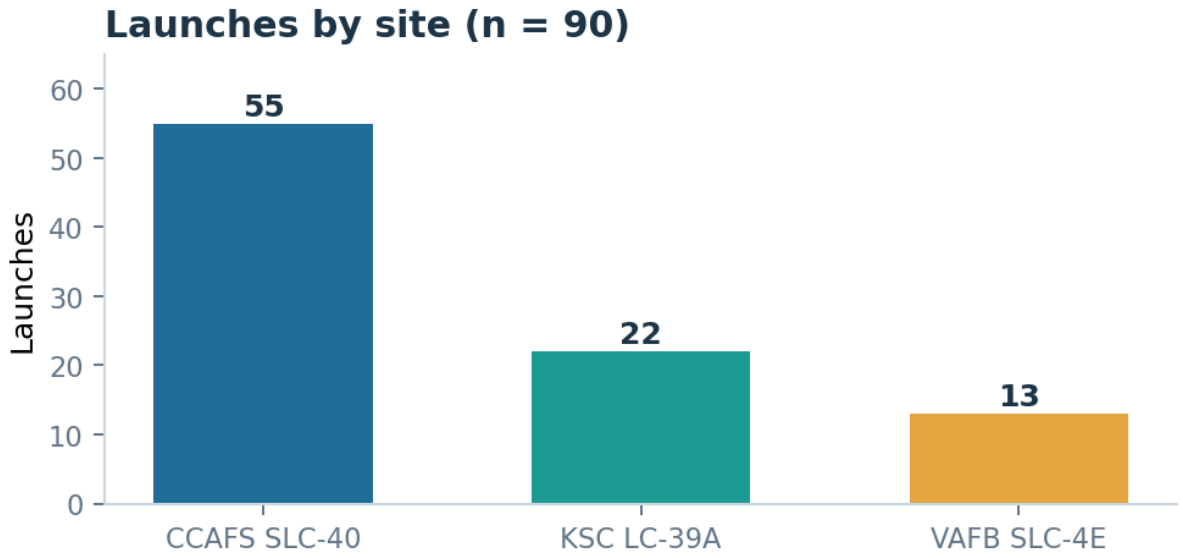
06 Split

Separate predictors and target; make train/test split before fitting models.

Leakage check: the landing outcome is the target, not an input feature. Fit scaling, encoding, imputation and hyperparameter tuning on training data only; evaluate final performance on held-out data.

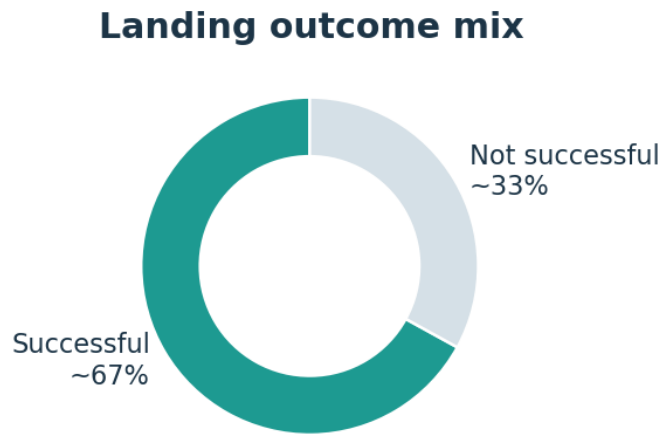
EDA with visualization: what the launch data shows

Counts and outcome mix give context before building predictive models



Launch-site counts

- CCAFS SLC-40: 55 launches
- KSC LC-39A: 22 launches
- VAFB SLC-4E: 13 launches



Interpretation

- The course dataset contains 90 records; the site counts sum to 90.
- Roughly two-thirds of outcomes are recorded as successful in the standard course snapshot.
- Launch-site counts are not success rates: compare success proportion within each site before drawing conclusions.

Additional lab plots examine flight number vs site, payload mass vs site, orbit vs flight number, payload vs orbit, and landing success over time.

EDA with SQL: validate patterns with queries

SQL provides reproducible counts and grouped summaries alongside Python charts

QUERY 1 | LAUNCH COUNT BY SITE

```
SELECT launch_site, COUNT(*) AS launch_count
FROM spacex_launches
GROUP BY launch_site
ORDER BY launch_count DESC;
```

WHAT THIS ANSWERS

Which site has the most Falcon 9 records? CCAFS SLC-40, with 55 in the 90-launch course dataset.

Follow-up SQL questions

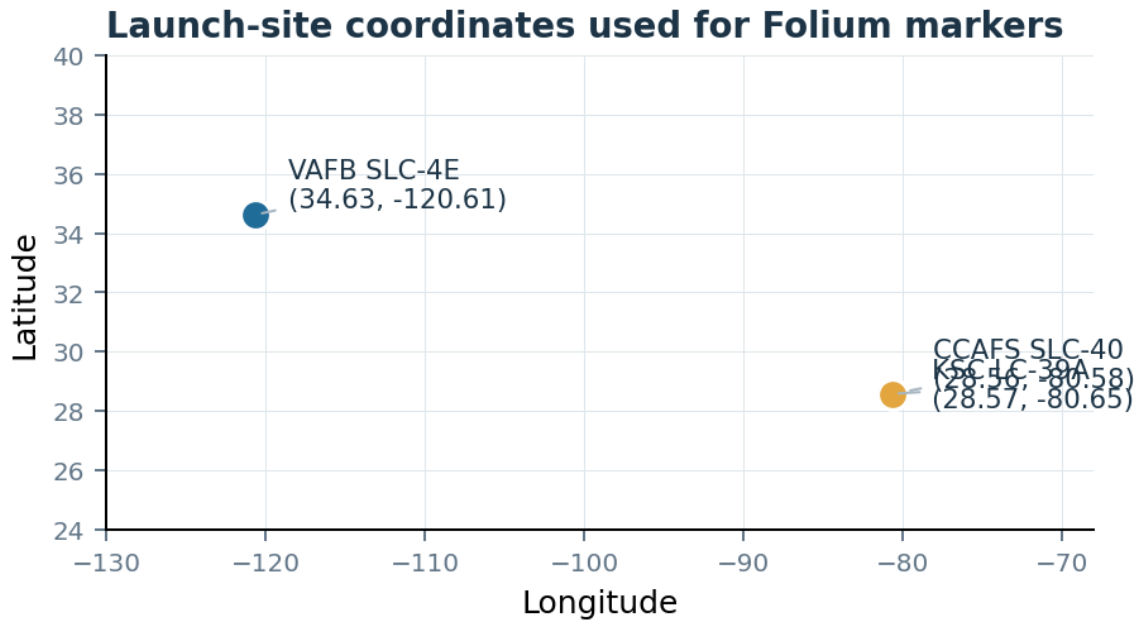
- How many distinct launch sites are present?
- Which orbit categories appear most often?
- What are minimum, maximum and average payload masses?
- How do landing outcome counts change by site or orbit?

Why SQL matters

SQL is useful for checking totals independently of Python, summarizing groups, and making it easier to reproduce the values shown in the final report.

Folium map: connect outcomes to geography

The interactive map locates launch facilities and adds context to site-level comparisons



Static locator based on launch-site coordinates

Map layers and interactions

- Marker / CircleMarker for each launch site
- Popup or tooltip with the site name and summary
- Marker color can encode a landing outcome or success proportion
- Cluster markers where points overlap

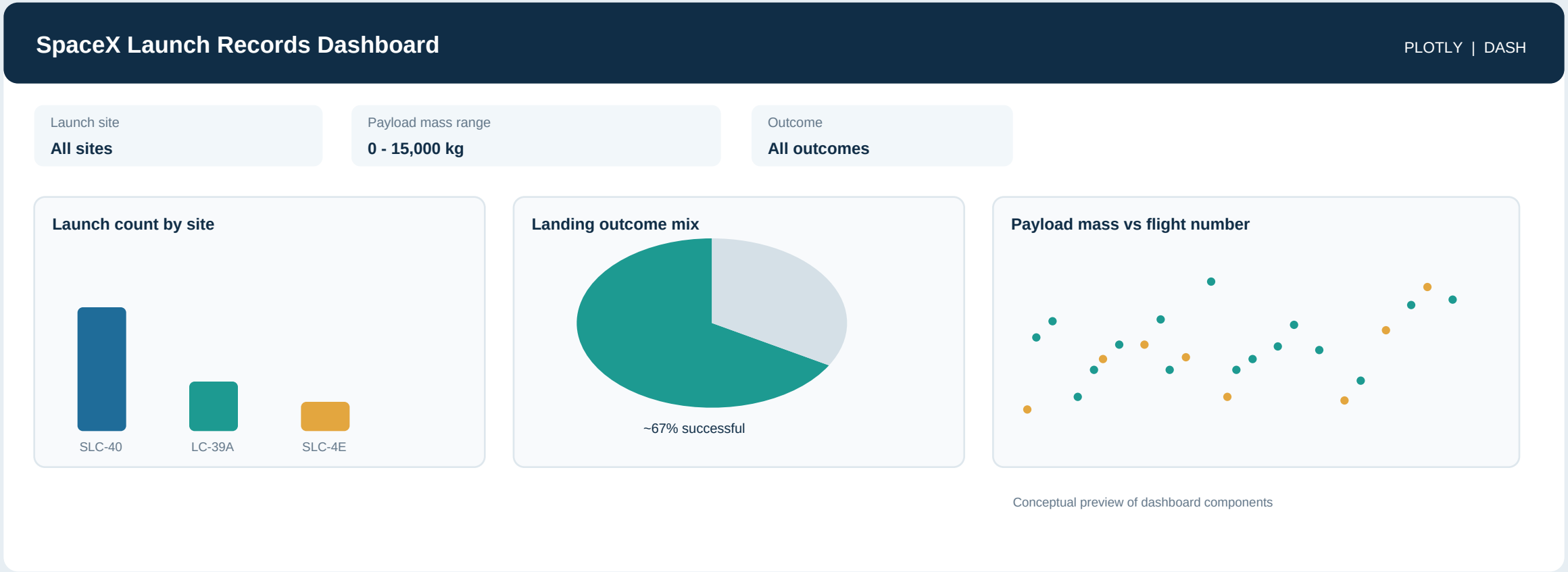
Questions explored

- Which launch sites are used?
- How are sites located relative to coastlines and nearby infrastructure?
- Do site-level success patterns differ?

This panel summarizes the Folium analysis with site coordinates; run the Folium notebook to view the interactive map and popups.

Plotly Dash: an interactive launch dashboard

A filter-driven interface makes the exploratory findings easier to use



Conceptual preview of dashboard components

Features: site dropdown, payload slider, live chart updates, site/outcome summaries. The actual Dash app should be run from the submitted Python file.

Classification methodology

Train, tune and evaluate four algorithms on the same landing-outcome target

1. Features

Launch site, flight number, payload mass, orbit and other available mission attributes

2. Preprocess

One-hot encode categories; scale numeric variables where required

3. Split

80% training / 20% test, stratified when appropriate

4. Tune

GridSearchCV with cross-validation to compare hyperparameters

5. Evaluate

Accuracy, confusion matrix, precision, recall and F1-score

MODELS COMPARED

- Logistic Regression
- SVM
- Decision Tree
- KNN

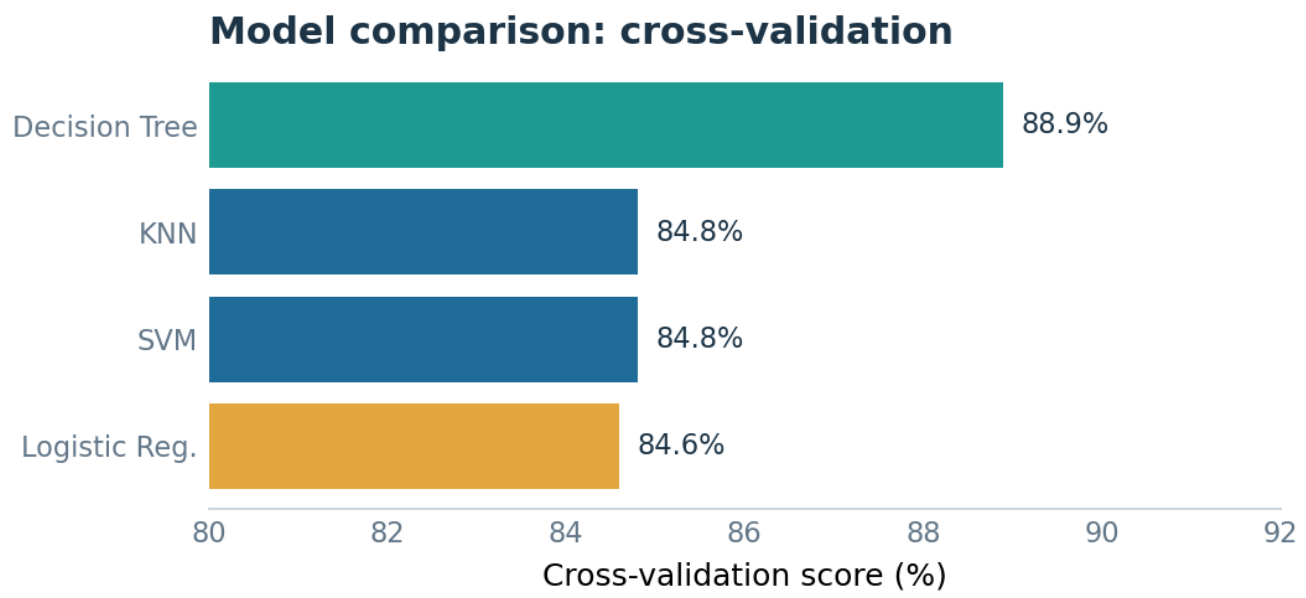
Evaluation principle

The held-out test set estimates generalization. Cross-validation is used for model selection. For a small dataset, report both and avoid claiming certainty from one split.

Target: first-stage landing outcome (successful vs not successful).

Predictive analysis: model comparison

In the standard course workflow, test accuracy ties; cross-validation separates the models



Cross-validation model ranking

Best cross-validation model

Decision Tree scored approximately 88.9% in the standard course implementation, ahead of KNN and SVM (~84.8%) and Logistic Regression (~84.6%).

Test-set interpretation

The common course run reports about 83.3% test accuracy for each model. Since the test scores tie, use cross-validation, confusion-matrix errors, and interpretability to choose a model.

Metrics to report from your executed notebook: best estimator and parameters; test accuracy; confusion matrix; precision; recall; F1-score. Small test sets make individual examples strongly affect the percentage.

What the capstone demonstrates

A reproducible end-to-end workflow from raw launch records to model comparison

Data insight

Launch records are concentrated at CCAFS SLC-40 (55/90). Landing outcomes vary by flight history, site, payload and orbit, so grouped comparisons help reveal patterns hidden in raw rows.

Analytics insight

SQL provides independent grouped counts; charts show distributions and relationships; Folium adds geographic context; Plotly Dash lets users filter and compare launch records.

Model insight

Decision Tree had the strongest cross-validation score in the standard run. Since small-sample test accuracy can tie across models, use multiple evaluation metrics and keep the workflow reproducible.

Limitations

- The dataset is small and reflects a historical snapshot.
- Launch design and operations evolve; historical patterns may not generalize to future missions.
- Some unsuccessful landings may be planned outcomes; labels require domain-aware interpretation.
- Predictions support analysis but do not replace engineering review.

Conclusion: data collection, careful wrangling, exploratory analysis, interactive visualization, and classification together create a useful decision-support workflow for studying first-stage recovery.

GitHub repository and submission checklist

The rubric specifically requires a repository URL with completed notebooks and Python files

Repository contents expected

- API data collection notebook
- Web scraping notebook
- Data wrangling notebook
- EDA with visualization notebook
- EDA with SQL notebook
- Folium map notebook
- Plotly Dash Python file and supporting data
- Machine-learning prediction notebook
- README with setup/run instructions

Repository URL - required action

Replace this placeholder with your own public GitHub URL before submitting:

[PASTE YOUR GITHUB REPOSITORY URL HERE]

Do not use another learner's repository as if it were yours. Make sure your files are uploaded and accessible.

Final upload checklist

- Upload this final presentation as a PDF.
- Put the real public GitHub repository URL on this slide and in any submission field that asks for it.
- Confirm all required notebooks and the Dash Python file are present in the repository.
- Make sure the model scores in these slides match the outputs in your executed notebook; update them if your results differ.
- Include meaningful explanatory content, visuals, query examples, and model results on every slide.

Important: the PDF is now in presentation format, but no PDF can guarantee a passing score. The real repository URL and matching notebook results are essential for the rubric.